

- [TechPrep AI — The Ultimate AI Interview Practice Platform](#)
 - [1. Introduction and Background](#)
 - [2. Requirements and Project Planning](#)
 - [3. System Design and Architecture](#)
 - [Component Architecture](#)
 - [4. Methodology and Model Development](#)
 - [5. Implementation and Testing](#)
 - [6. Results, Analysis and Discussion](#)
 - [7. Deployment](#)
 - [8. Conclusion and Future Work](#)

TechPrep AI — The Ultimate AI Interview Practice Platform

1. Introduction and Background

TechPrep AI is a full-stack, production-ready interview practice platform designed to help computer science students and software engineers prepare for technical and behavioral job interviews. With the rising competitiveness in the tech industry, candidates often lack realistic, highly-structured interview practice. Traditional mock interviews are difficult to schedule, and generic AI chatbots do not offer the rigorous, domain-specific deep dives required to ace top-tier technical interviews.

To address this gap, TechPrep AI serves as an automated, highly knowledgeable AI Interviewer. The platform generates customized questions based on user roles and specific domains, scores answers across 6 unique dimensions, streams real-time coaching feedback using Server-Sent Events (SSE), and provides granular technical corrections just like a real human interviewer.

2. Requirements and Project Planning

The project was planned with a focus on realism, low-latency interaction, and actionable feedback. The core requirements were:

- **Role and Domain Customization:** The ability to target specific roles (e.g., Backend SDE, Machine Learning Engineer) and lock into specific domains (e.g., Database Design, Algorithms) to mimic the structured rounds of companies like HackerRank or FAANG.
- **Realistic Feedback Loop:** The AI must not only evaluate answers but also provide real-time, spoken-style verbal corrections, identify factual errors, highlight missed key concepts, and provide an ideal answer outline.
- **Interactive Voice & Chat Coaching:** Support for fully interactive voice-mode where the AI coach speaks to the user using Web Speech Synthesis and listens to responses.
- **Comprehensive Evaluation:** Grading answers instantly on STAR Structure, Technical Depth, Communication, Relevance, Confidence, and Conciseness.
- **Performance Tracking:** A dashboard to visualize streaks, previous sessions, and performance trends over time using asynchronous PDF reports.

3. System Design and Architecture

TechPrep AI utilizes a modern, decoupled microservices architecture to ensure scalability and responsiveness during real-time interactions.

Component Architecture

- **Frontend (React/Vite):** A Single Page Application handling real-time state management, audio synthesis, and SSE streaming. Uses Zustand for state, TanStack Query for data fetching, and Recharts for data visualization.
- **Backend API (Node.js/Fastify):** The primary gateway for user authentication, session management, and database operations. It securely handles OAuth and stores interview records.
- **AI Service (Python/FastAPI):** A dedicated Python microservice that interfaces directly with Google's Gemini 1.5 Pro. It manages complex prompt engineering, JSON-enforced schema generation, and asynchronous tasks like PDF report generation via ReportLab and Matplotlib.
- **Database & Queues:** PostgreSQL serves as the primary relational database (managed via Prisma ORM), while Redis manages caching and background job queues (via ioredis).

4. Methodology and Model Development

The core intelligence of TechPrep AI is driven by Google Generative AI (Gemini 1.5 Pro). The methodology for model interaction relies heavily on **Structured Prompt Engineering**.

- **Role-Specific Competency Maps:** The AI uses predefined domain lists and competency maps. During question generation, the model is injected with a `selected_domain` constraint, forcing it to generate strict, topic-based questions rather than generic filler.
- **Granular Output Schemas:** The system enforces strict JSON outputs from the LLM. The evaluation prompt dictates that the model must return not just a numerical score, but a detailed `ScoreAnswerResponse` object containing:
 - `technical_errors`: Specific coding or architectural mistakes.
 - `key_concepts_missed`: Important topics the candidate forgot to mention.
 - `what_was_correct`: Positive reinforcement of correct statements.
 - `interviewer_correction`: A conversational, spoken-style correction as if the interviewer is interrupting to correct the candidate.
 - `ideal_answer_outline`: A structured blueprint of the perfect response.

5. Implementation and Testing

Implementation was carried out across the entire stack:

- **Database:** A `selectedDomain` column was added to the `Session` table in PostgreSQL to persist the context of the interview.
- **API Layer:** Fastify routes were updated to accept domain parameters and forward them to the Python AI service.
- **AI Service:** The `role_prompts.py` file was heavily modified to support domain locking and granular feedback extraction. The integration was tested using `run_gemini_json` to ensure the LLM adhered to the strict JSON constraints.
- **Frontend UI:** The React application was enhanced with a domain selector during session creation. The `SummaryPage` and `SessionPage` were updated to render the new detailed technical breakdown in beautifully styled accordions and real-time chat bubbles.

6. Results, Analysis and Discussion

The implementation of domain locking and granular feedback fundamentally transformed the platform from a generic conversational agent into a rigorous technical interviewer.

Analysis of the changes:

- **Targeted Practice:** Users can now focus entirely on their weak areas (e.g., System Design) without being derailed by unrelated behavioral questions.
- **Actionable Feedback:** Previously, the AI provided generic coaching advice. Now, candidates receive highly specific corrections, directly pointing out flaws in their logic or missing constraints in their algorithms.
- **Realism:** The addition of the "Interviewer's Verbal Correction" bubble mimics the natural flow of a real interview, where an interviewer might gently correct a candidate before moving on to the next question.

7. Deployment

TechPrep AI is designed for modern serverless and containerized deployment environments.

- **Frontend:** Deployed to Vercel, providing edge caching and fast global delivery.
- **Node.js API:** Deployed as a Web Service to Render.
- **Python AI Service & Workers:** Deployed as Web Services and Background Workers on Render, handling heavy ML and PDF processing tasks asynchronously.
- **Database & Redis:** Supabase Postgres and Upstash Redis clusters ensure high availability and low latency data access.
- **Local Development:** The entire stack is orchestrated locally using `docker-compose`, providing a seamless developer experience with hot-reloading.

8. Conclusion and Future Work

TechPrep AI successfully demonstrates the capability of modern LLMs to serve as highly effective, domain-specific technical interviewers. By enforcing strict schemas

and domain constraints, the platform provides an invaluable tool for candidates preparing for competitive tech roles.

Future Work:

- **Code Execution Environment:** Integrating a live code editor where the AI can compile and run candidate code, providing feedback on syntax and algorithmic complexity in real-time.
- **Adaptive Difficulty:** Implementing an algorithm to dynamically adjust the difficulty of subsequent questions based on the candidate's performance on the current question.
- **Expanded Domain Libraries:** Partnering with industry experts to build even more rigorous competency maps and grading rubrics for highly specialized roles like Quantum Computing or Security Engineering.